

Zero-Trust Architecture for Agentic AI

Securing Multi-User Access and Third-Party Integrations

Prairie Dev Con 2026

Borko Djurkovic

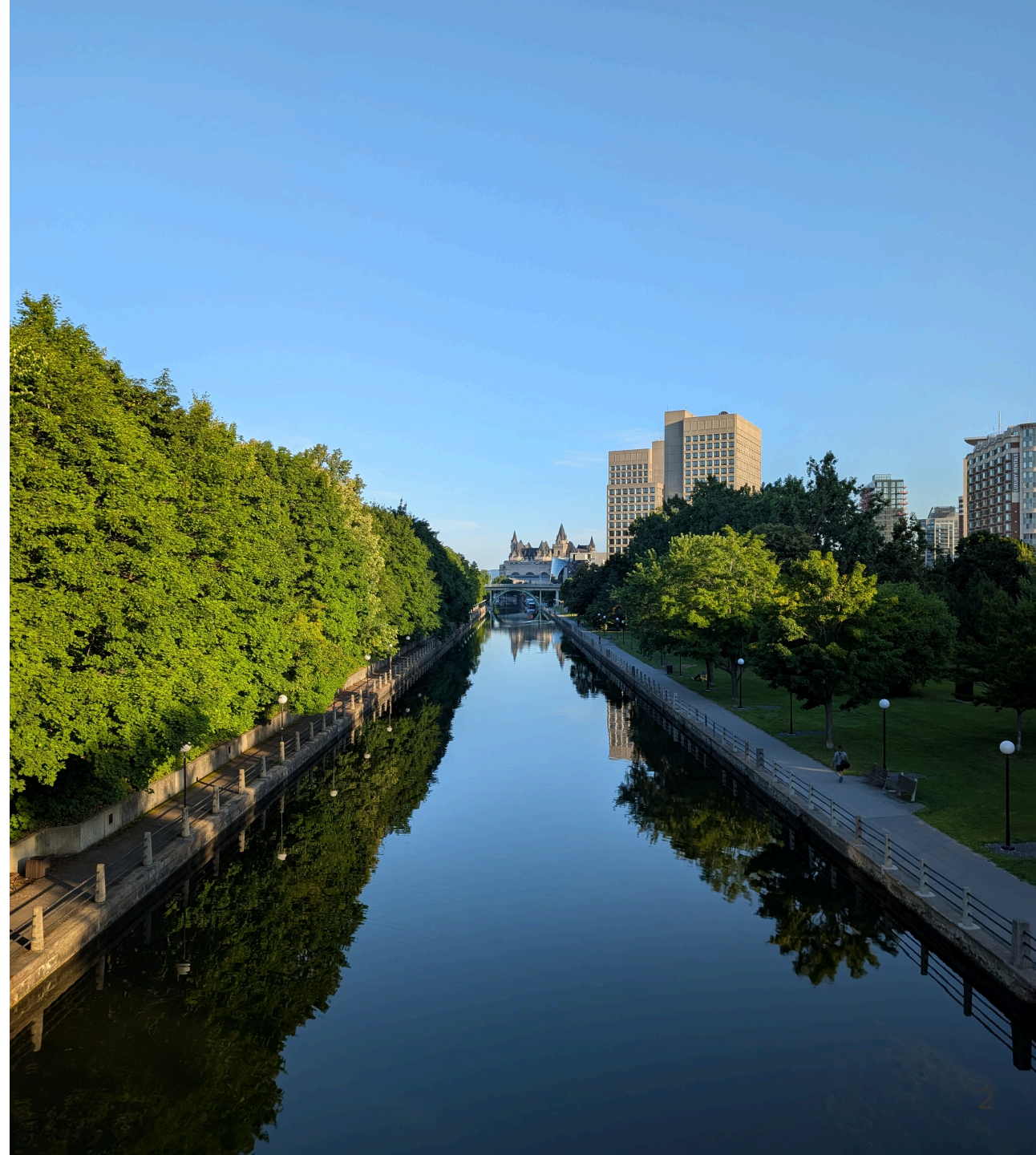
ABOUT ME

 Software Engineer

 Ottawa, Canada

 Runner

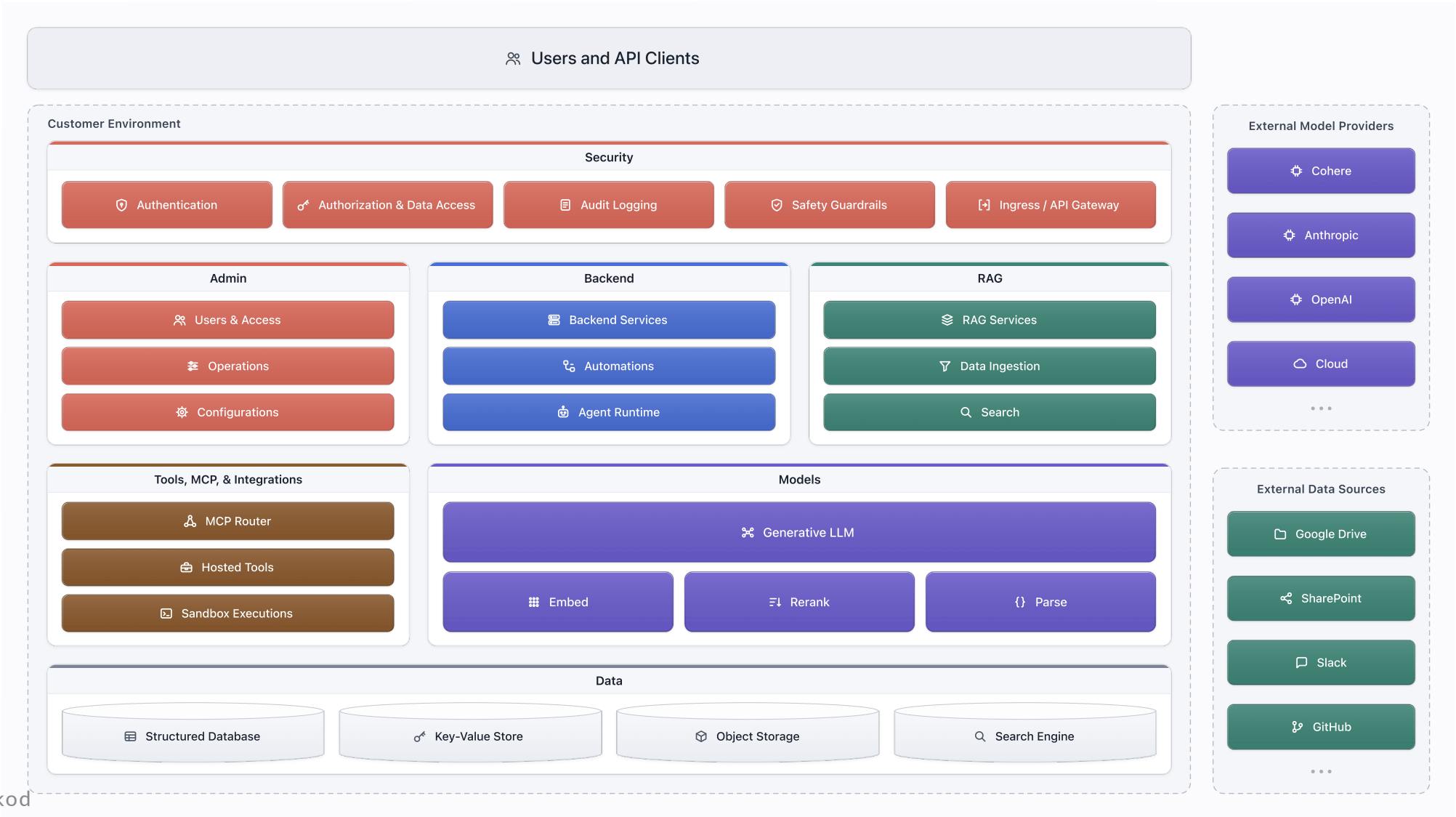
 Podcast Listener



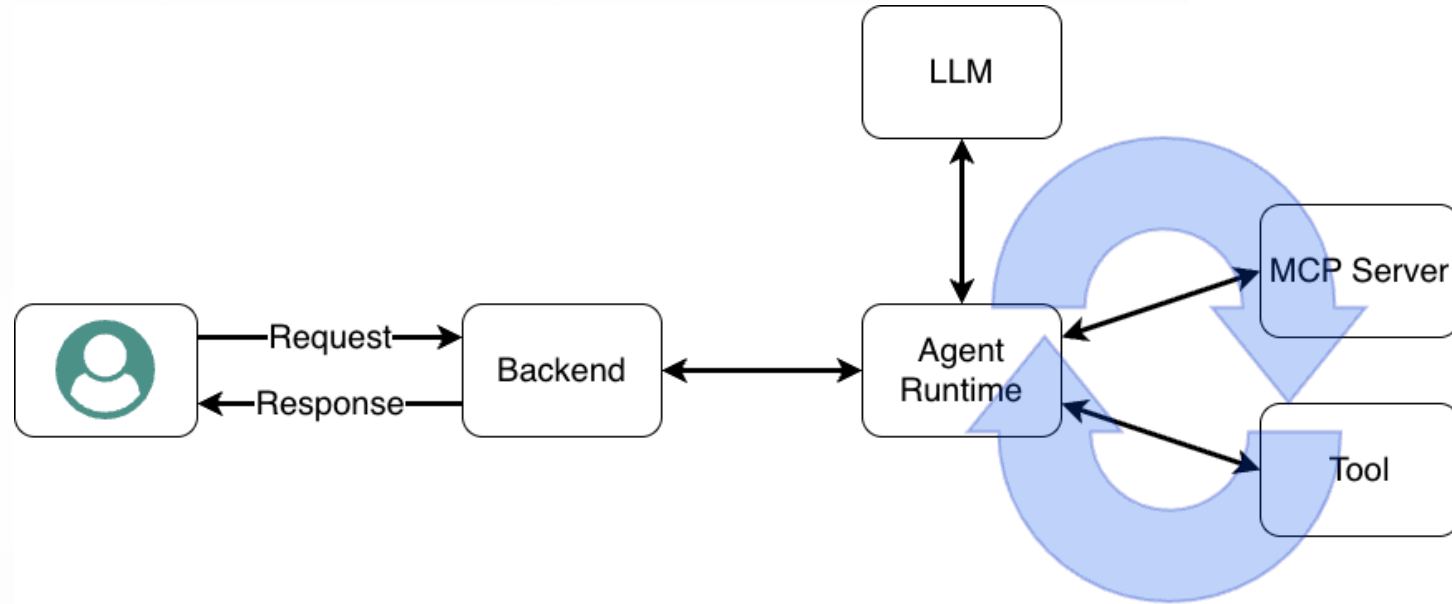
Key Takeaways

- **Identity & Access:** Authenticate users through OIDC and enforce least-privilege access to platform resources using RBAC and ReBAC.
- **Tool & Integration Security:** Authenticate and authorize every external tool call, preferring per-user delegation where downstream services need to enforce user permissions.
- **Secure RAG:** Preserve source-system ACLs through ingestion and enforce them during retrieval so users only receive content they are authorized to access.

Agentic AI Platform



Typical Request



The agent runtime is responsible for executing the agent on the user's behalf.

The Need for RBAC

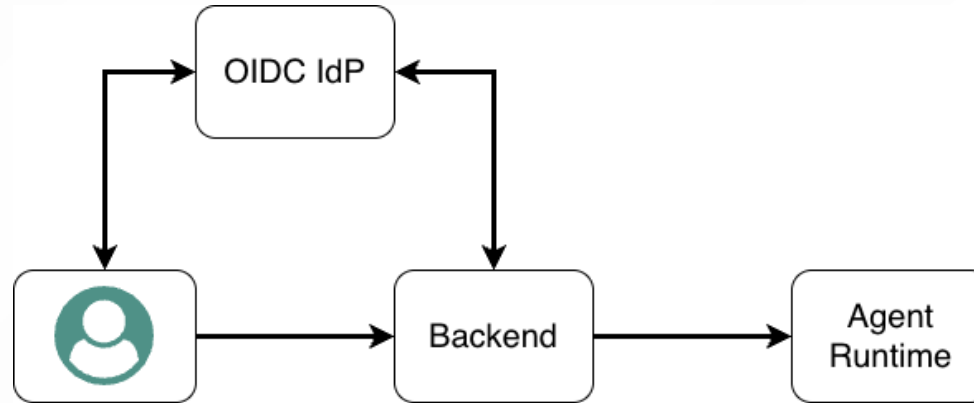
An enterprise AI platform may have thousands of users with very different responsibilities.

Who they are	Should be able to...	Should not be able to...
Platform Admin	Configure model providers, connectors, feature settings	Read user conversations
Agent Developer	Build agents, edit prompts, configure tools, publish to a team	Change data retention settings
End User	Use approved agents & tools	Change models or tools
Auditor	Read logs, runs & access history	Modify anything

Enterprise access is about least privilege: people should have the access they need and no more.

Role-Based Access Control (RBAC) assigns permissions to roles, then assigns users to those roles.

OIDC: Establishing Identity



What OIDC Provides:

- Authentication: establish who the user is
- Identity claims: subject ID, email, name, groups, etc.
- OIDC tokens: short lived
- SSO: users authenticate through the organization's existing IdP
- Users identity claims (e.g. groups) map to RBAC roles
- Centralized identity lifecycle: accounts are managed by the organization's IdP

What an AI Platform Has to Authorize

RBAC works well for stable capabilities. But an enterprise AI platform creates a large and constantly changing set of resource instances.

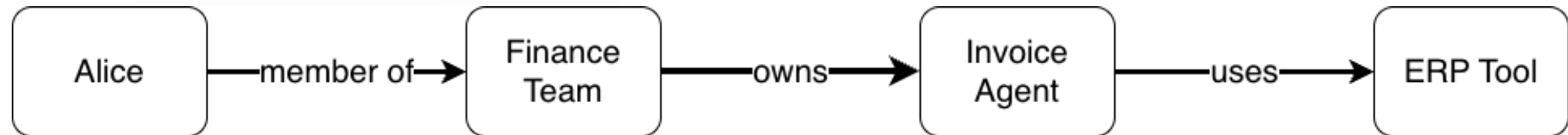
Resource	Examples	Authorization questions
Agents	Invoice agent, HR agent, RFP drafter	Who can use or modify this agent?
Models	Command A+, Claude, GPT, Kimi	Who can use this model?
Tools & MCP servers	Salesforce, SharePoint, Jira, Linear	Which agents/users can invoke this tool?

Where RBAC gets difficult

- Role permissions describe resource *types*, but real authorization is often about *instances*
 - *Which agent? Which tool? Which data source?*
- Scoped roles help, but can lead to role explosion
 - *Who can access this document?*
- Context can matter beyond the role
 - *Can this agent invoke this particular tool for this particular user?*

ReBAC: Access Through Relationships

- Relationship-Based Access Control (ReBAC) makes authorization decisions based on relationships between entities.
 - Authorization becomes a query over the platform's entity graph.
- Instead of asking:
- *Is Alice an AgentDeveloper?*
- ReBAC asks:
 - *Is Alice a member of the team that owns this agent?*
 - *Is this agent allowed to use this tool?*
 - *Can this agent invoke this particular tool for this particular user?*



OpenFGA: Relationship-Based Authorization

OpenFGA is an open-source authorization system for modeling and evaluating fine-grained permissions through relationships.

OpenFGA represents these relationships as tuples:

```
team:finance#member@user:alice  
agent:invoice#owner@team:finance  
tool:ERP#can_use@agent:invoice  
provider:cohere#can_use@user:alice
```

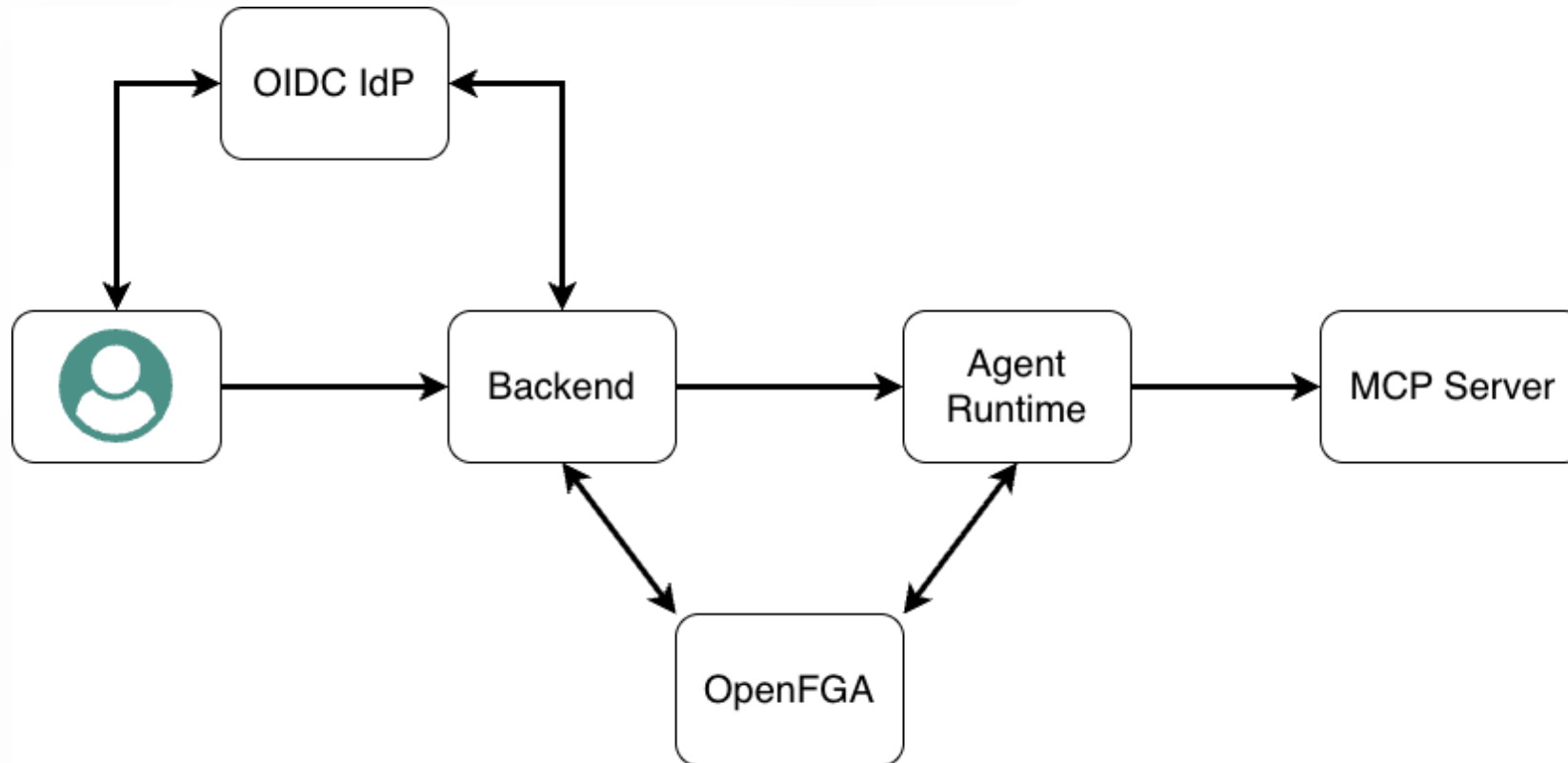
Why it fits Agentic AI:

- Fine-grained: access to agents, tools, knowledge bases, etc
- Relationship-aware: ownership, membership, sharing, inheritance
- Centralized: authorization logic is defined outside application code
- Auditable: relationships and authorization policy are explicit
- Separates application logic from authorization policy
- Scales with resource relationships rather than requiring a role for every resource

OpenFGA: Relationship-Based Authorization

OpenFGA evaluates the authorization model and relationships: *Can this principal perform this action on this resource?*

The application owns the resources and enforces the authorization decision.



OpenFGA Pitfall: Keeping Two Systems in Sync

Creating or changing a resource is now a **multi-system operation**. Database can succeed while the corresponding OpenFGA update fails.

Two systems, one lifecycle:

- **Database owns resource state**
 - The agent/provider/tool exists
 - Configuration and metadata
 - Resource lifecycle state
- **OpenFGA owns authorization state**
 - Who can access the resource
 - Which users/agents can use it
 - Relationships between resources

Design for consistency

- **Don't hide OpenFGA failures** — retry and surface provisioning failures
- **Make writes idempotent** — retries must be safe
- **Reconcile continuously** — detect and repair missing relationships

OpenFGA Pitfall: Stale Authorization Decisions

OpenFGA can cache authorization decisions to reduce latency and datastore load. Permission changes are not necessarily immediately visible

Caching introduces a tradeoff

Lower latency

- Cache authorization decisions
- Reduce datastore load
- Accept a bounded stale-decision window

Stronger freshness

- Evaluate against more current authorization state
- Higher latency / datastore cost
- Appropriate for sensitive operations and revocation

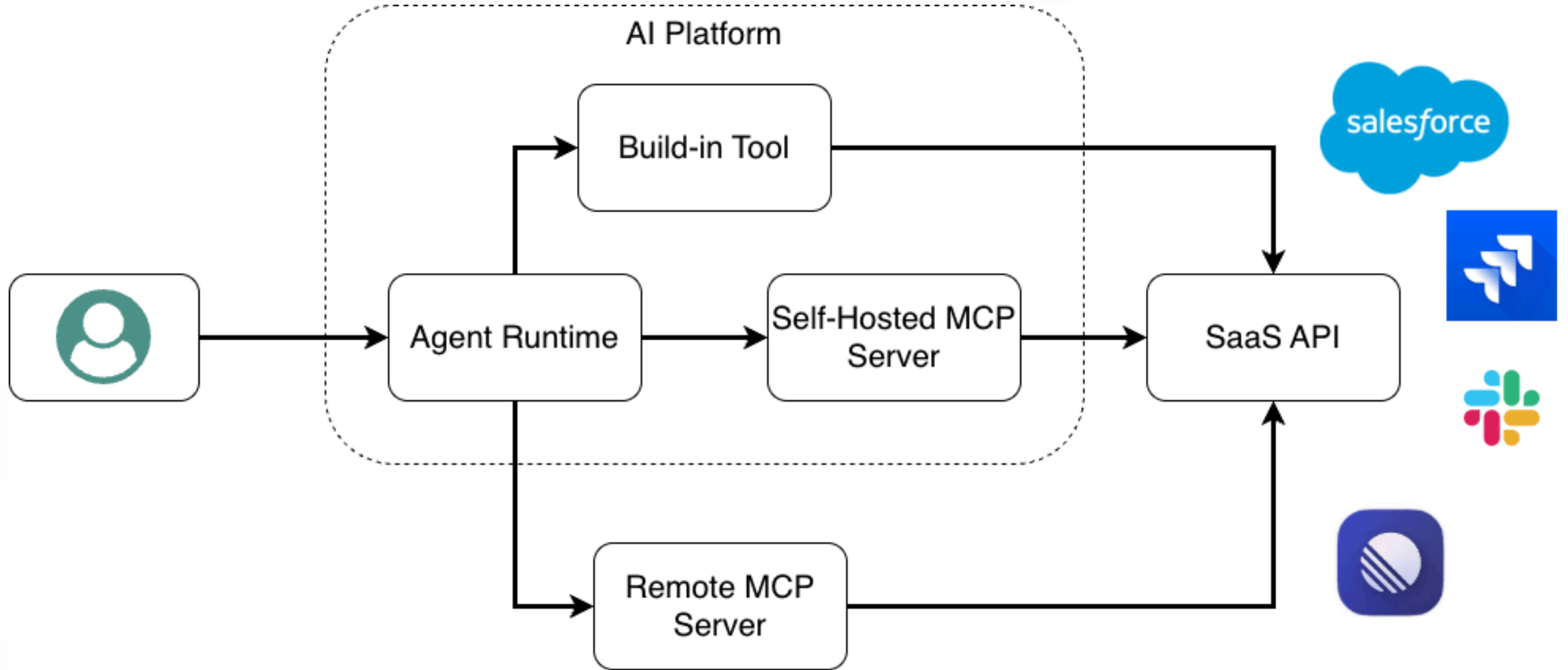
Make the tradeoff explicit

- Define how quickly permission revocation must take effect
- Set cache and consistency behavior accordingly
- Apply stronger guarantees to high-risk operations
- Test that revocation actually takes effect within the required window

Key Takeaways

- **Identity & Access:** Authenticate users through OIDC and enforce least-privilege access to platform resources using RBAC and ReBAC.

How Agents Reach Third-Party Systems



Authentication Patterns for Tool Calls

Platform Credential

- One identity for the AI platform: a Static API key, service account, or OAuth 2.0 client credentials
 - The external service sees: *The platform*
 - All users share the platform's access; user-level ACLs cannot be enforced
 - *Use when* the target has no concept of your users: an internal microservice, a public data API

Per-User Delegated Authorization

- The user signs in to the external service once through an OIDC/OAuth connector; the platform then calls with that user's token
 - The external service sees: *The user*
 - The external system's existing user permissions apply
 - *Use when* the external system already knows your users and enforces its own permissions: SharePoint, GitHub, Jira

Token Exchange

- An intermediary (e.g. MCP server) exchanges a caller's token for a new, short-lived token targeted at a downstream service
 - The resulting token preserves the user's authority while identifying the calling service as the actor
 - The external service sees: *The user + the calling service*
 - Enables scoped, audience-specific credentials without giving the intermediary the user's original credential
 - *Use when* a downstream service (e.g. an MCP server) must act on the user's behalf across a service boundary

Governing the Tool Surface

MCP provides a standard way to expose tools to the LLM. The platform still decides which tools are trusted, who can invoke them, and how they authenticate.

Control what can be called

- Approved registry: only admin-registered MCP servers can be connected
- Explicit enablement: tools are unavailable unless explicitly enabled for an agent/team

Control who can call it

- Least privilege: grant users and agents only the tools they need
- User authorization: use RBAC
- Use ReBAC: the tool becomes part of the authorization graph in OpenFGA

Control how it authenticates

- Prefer short-lived credentials: avoid long-lived secrets in tool configuration
- Require delegated credentials when the external service must enforce user permissions
- TLS everywhere: protect every network hop; use mutual TLS where appropriate

Governing the Model Surface

The platform should curate the model surface just as it curates the tool surface. Agents should never be free to select arbitrary models or providers.

Control what can be used

- Approved registry: only platform-vetted models/providers are available
- Explicit enablement: models must be enabled for an agent, team, or workload
- Consider model versions where reproducibility and change control matter
- Remove models when security, compliance, or provider risk changes

Control who can use it

- Least privilege: expose only the models an agent actually needs
- RBAC/ReBAC: model access is part of the authorization graph
- Separate access by sensitivity or purpose: e.g. general-purpose, confidential-data, regulated workloads
- Enforce organizational policy before a model request reaches the provider

Control how it is accessed

- Platform-managed provider credentials
- Centralize egress, logging, rate limits, quotas, and cost controls

Models Are Different From Tool Calls

A tool call grants an agent the ability to *do something*. A model grants the agent the ability to *reason, generate, and transform information*. The security boundary is therefore different.

Tool / MCP	Model
Usually an explicit action	Generates the next action, content, or decision
Authorization is often about <i>what may be done</i>	Authorization is about <i>what capabilities may process the data</i>
Downstream system can enforce user permissions	Provider may have no knowledge of the enterprise user
Request/response is relatively deterministic	Behavior is probabilistic and model-dependent
Primary risk: unauthorized action	Primary risks: data exposure, unsafe output, policy bypass, model behavior

The Model Call Is a Data Egress Event

A model call sends your **context**: the retrieved chunks, conversation history, and tool output.

Data leaves our security boundary

- *Where* it is processed — data residency
- *Whether* it is retained: logging, monitoring
- *Whether* it is trained on
- *Who* at the provider, and which subprocessors, can access the data

What self-hosting changes

- The boundary becomes infrastructure you already operate: your VPC, network policy, IAM, audit log
- Residency becomes a deployment decision
- Retention is yours, rather than a contract clause
- The model sits inside the same perimeter as the index it reads from

A contract is an **obligation**. Infrastructure you control is an **enforcement mechanism**. Self-hosting converts a legal control into an architectural one.

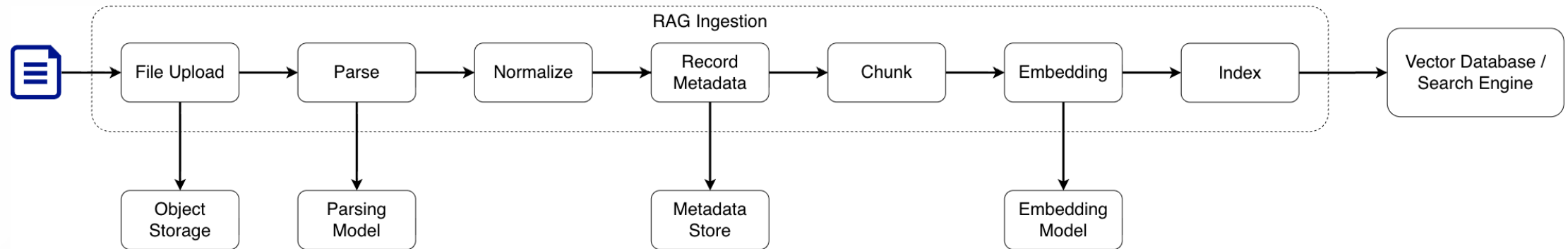
- **The cost:** capacity, scaling, model updates
- **Choose the model per workload:** let data classification and use case decide what model to use, rather than adopting one model or provider for the entire platform

Key Takeaways

- **Identity & Access:** Authenticate users through OIDC and enforce least-privilege access to platform resources using RBAC and ReBAC.
- **Tool & Integration Security:** Authenticate and authorize every external tool call, preferring per-user delegation where downstream services need to enforce user permissions.

RAG System: Ingestion

Transforming enterprise content into a searchable knowledge base



- **Parse:** Turn each format into structured content
- **Normalize:** Convert into a canonical representation
- **Chunk:** Split content; Each chunk becomes a unit of retrieval
- **Embed:** Produce a vector representation of its meaning
- **Index:** Write chunks, vectors, and metadata to the search engine; Index for both semantic similarity and keyword search

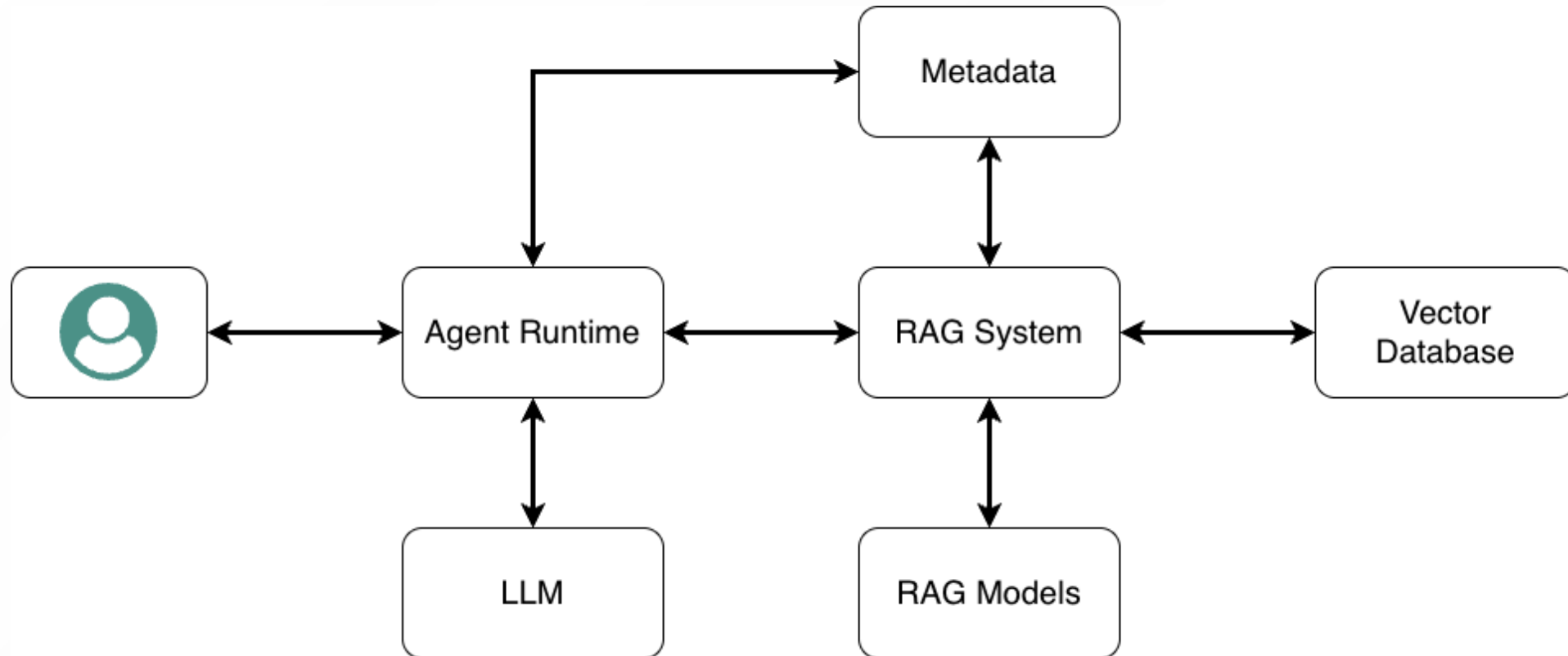
RAG Security: Data Curation at Ingestion

Treat the RAG corpus as a security boundary; not a copy of your source system.

- **Curate:** only ingest content the LLM application is allowed to consume
- **Redact:** remove sensitive fields before chunking and embedding
- **Don't rely on retrieval filtering:** once sensitive content is embedded, it has already entered the retrieval system
- **Preserve the source of truth:** the RAG corpus is a purpose-built representation, not a replica of the original documents

Example: A customer document contains SSNs + product information → the RAG pipeline removes the SSNs → only the product information is embedded.

Typical Request with RAG



Securing RAG Retrieval

Semantic relevance is not authorization.

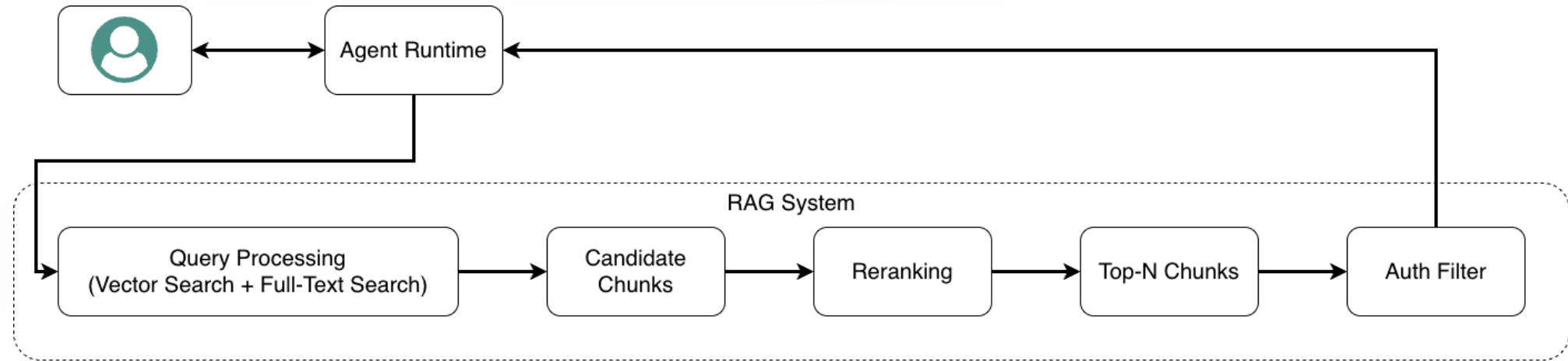
A vector search finds the most relevant chunks, *even when the user isn't allowed to see the source document.*

Layer	Question	Mechanism
Knowledge Base / Index	Can this user search this corpus?	RBAC / ReBAC
Document	Can this user access this document?	Source ACL
Chunk	Can this chunk be returned?	Inherited document ACL

Enforce access at retrieval time:

- **At ingest:**
 - Extract the source system's ACL; Store authorization metadata with every chunk
- **At query:**
 - The user's group memberships become an authorization filter *inside* the search; Match user's groups to chunk's ACL
 - Return only chunks the caller is authorized to access

Securing RAG Retrieval



Authorization must be part of retrieval. If an unauthorized chunk reaches the application or LLM, it is already too late.

Key Takeaways

- **Identity & Access:** Authenticate users through OIDC and enforce least-privilege access to platform resources using RBAC and ReBAC.
- **Tool & Integration Security:** Authenticate and authorize every external tool call, preferring per-user delegation where downstream services need to enforce user permissions.
- **Secure RAG:** Preserve source-system ACLs through ingestion and enforce them during retrieval so users only receive content they are authorized to access.

END